Technical Report: Image Classification on CIFAR-10

Author: Francisco Manuel Romero Lobato

Role: AI Engineer / Solutions Architect

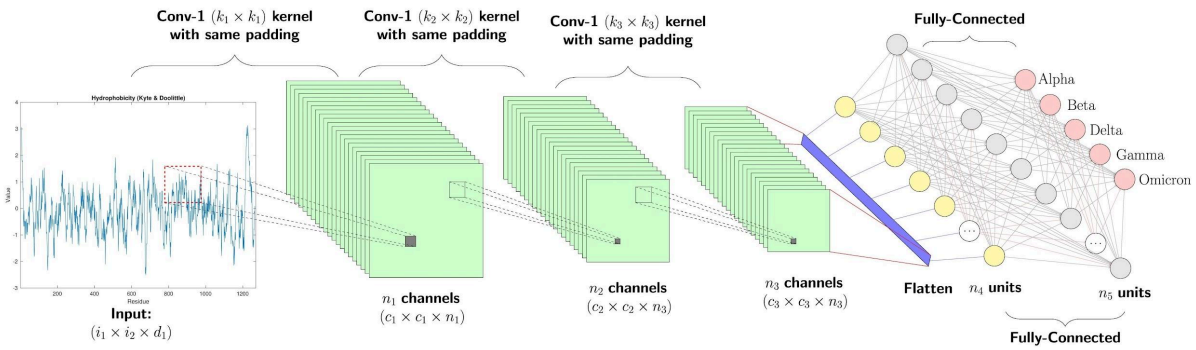
Project: Benchmarking CNN Architectures on CIFAR-10

---

1. Chosen CNN Architecture Description

For the baseline model, a **Custom CNN** was designed using the Keras Sequential API. The architecture follows a classic feature extraction pattern followed by a classification head:

- **Convolutional Blocks:** Two primary blocks. The first contains 32 filters and the second 64 filters, both utilising (3,3) kernels with **ReLU** activation.
- **Spatial Reduction: MaxPooling2D** layers with a (2,2) window were applied after each block to reduce spatial dimensions and concentrate the most relevant features.



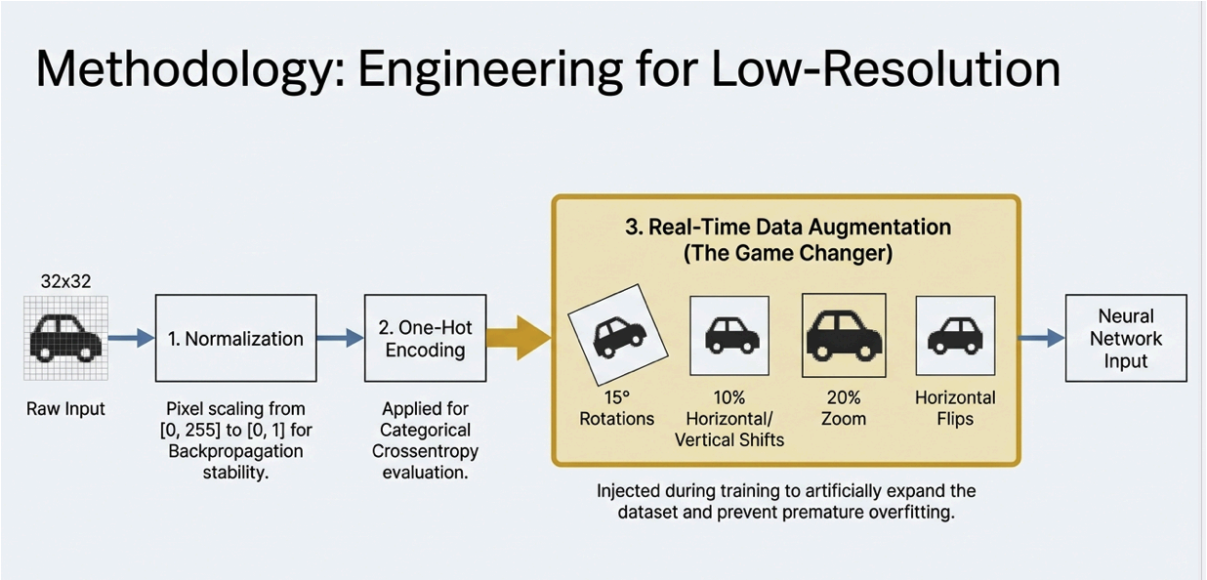
Shutterstock

- **Regularisation: Dropout** layers were integrated (0.25 within convolutional blocks and 0.5 in the dense layer) to mitigate **Overfitting** and enhance the model's generalisation capabilities.
- **Classification Head:** A **Flatten** layer converts the 2D feature maps into a 1D vector, followed by a **Dense** layer with 512 neurons and a final 10-neuron output layer with **Softmax** activation.

2. Preprocessing Steps

The CIFAR-10 dataset, consisting of 60,000(32,32) pixel images, underwent several critical preprocessing stages:

- **Normalisation (Feature Scaling):** Pixel values were rescaled from a [0, 255] range to [0, 1] by dividing by 255.0. This ensures numerical stability during the **Backpropagation** process.
- **One-Hot Encoding:** Integer class labels were transformed into binary matrices (10 columns), a requirement for using **Categorical Crossentropy** as the loss function.
- **Data Augmentation:** For the custom model, an ImageDataGenerator was utilised to perform real-time transformations, including 15° rotations, 10% horizontal/vertical shifts, 20% zoom, and horizontal flips. This artificially increased the diversity of the training set and improved robustness.



3. Training Process Details

The training parameters were standardised to ensure a fair comparison, utilising the **Adam** optimiser for its adaptive learning rate capabilities:

Parameter	Custom CNN	Transfer Learning Models
Optimizer	Adam	Adam
Learning Rate	0.001	0.001
Loss Function	categorical_crossentropy	categorical_crossentropy
Batch Size	128	128 (64 for InceptionV3)
Max Epochs	100	20 - 50
Early Stopping	Patience: 10 epochs	Patience: 5 epochs

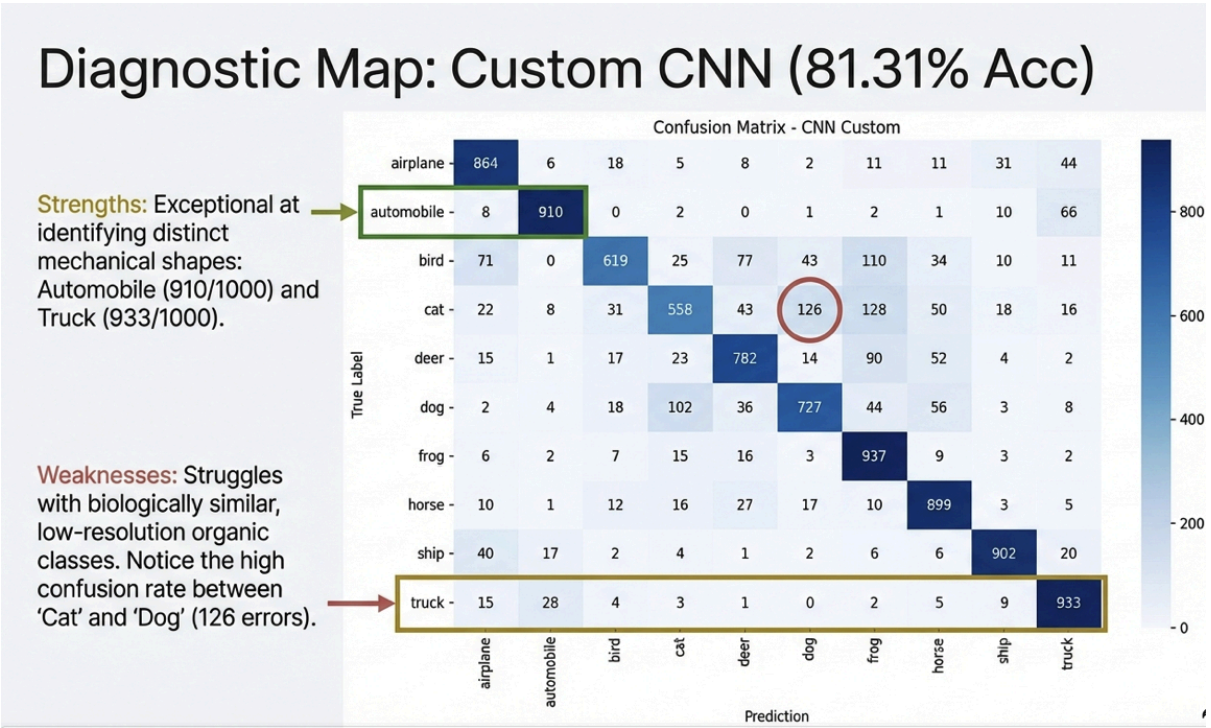
4. Results and Performance Analysis

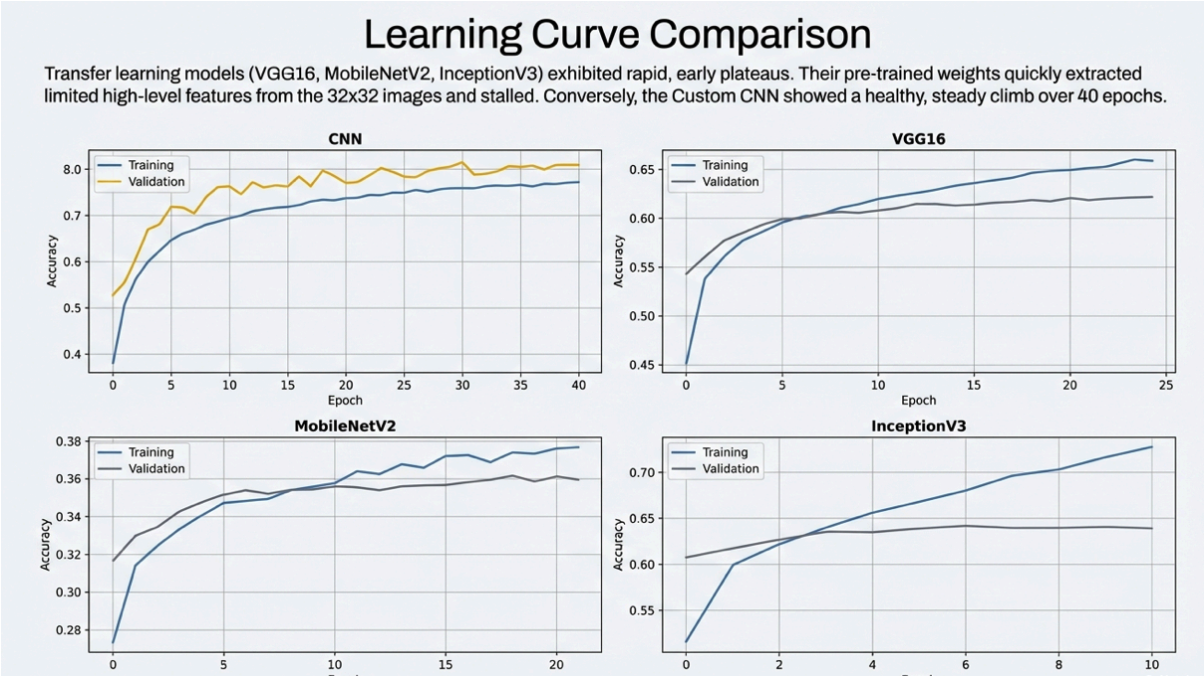
After evaluating the models on the 10,000-image test set, the final **Accuracy** results were as follows:

1. **Custom CNN: 81.31%**
2. **InceptionV3:** 63.91%
3. **VGG16:** 62.06%
4. **MobileNetV2:** 35.81%

Per-Class Analysis (Custom CNN):

The custom model showed exceptional performance in identifying categories such as **Automobile (92%)** and **Ship (91%)**. However, it faced challenges with biologically similar classes, specifically **Cat (56%)** and **Bird (62%)**, highlighting the difficulty of capturing complex organic textures at low resolutions.

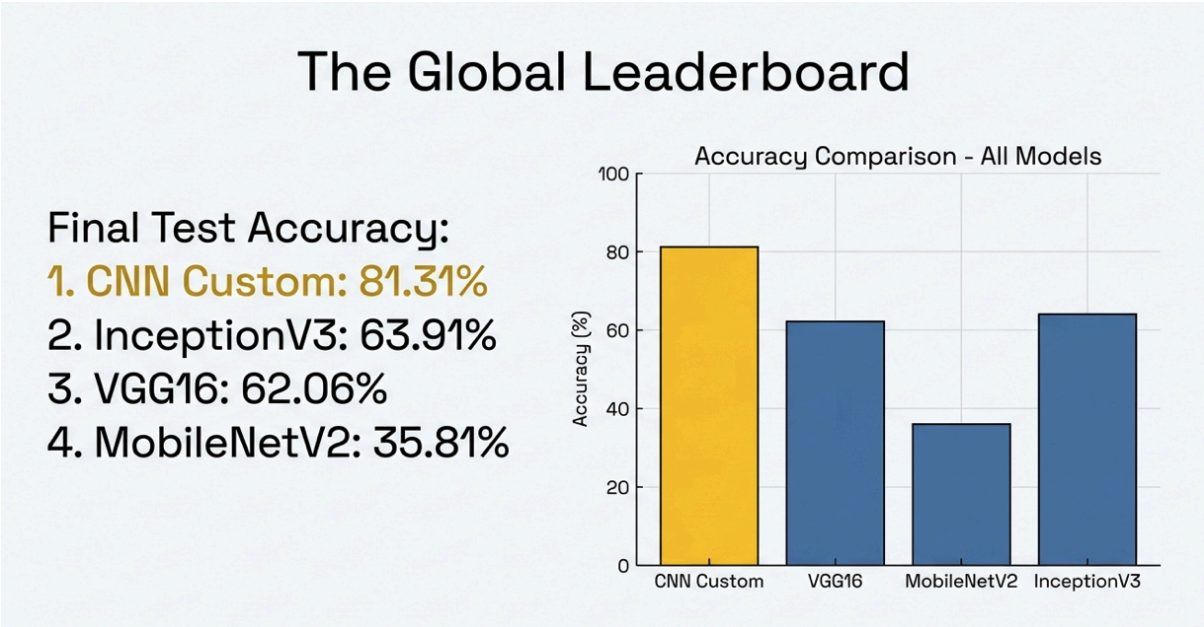




---

5. The Best Model: Custom CNN

According to the experimental results, the **Best Model is the Custom CNN** with an accuracy of **81.31%**.



Why?

- **Domain Effectiveness:** Despite not using pre-trained weights from ImageNet, the implementation of **Data Augmentation** allowed the model to learn robust features specifically tailored to the native (32,32) CIFAR-10 resolution.
- **Efficiency:** It is a lightweight architecture with significantly faster training times and a smaller memory footprint compared to heavyweight models like VGG16.
- **Metric Superiority:** It outperformed the second-best model (InceptionV3) by nearly 20 percentage points, proving that for this specific low-resolution dataset, a specialised architecture was more effective than a general-purpose deep model without extensive **Fine-tuning**.

---

6. Insights Gained

- **Data Augmentation is Vital:** The high performance of the custom CNN was largely due to the augmentation strategy, which effectively prevented premature overfitting.
- **Transfer Learning Limitations:** Models like InceptionV3 require dynamic resizing (75,75), which may introduce artefacts or data loss. This suggests that the "frozen" features from ImageNet (trained on high-res (224,224) images) do not always translate perfectly to low-res datasets without further optimisation.
- **Memory Management (OOM):** Implementing dynamic resizing layers and reducing batch sizes for complex models were key architectural lessons to avoid **Out-of-Memory (OOM)** errors in local environments.